

Trabalho Final

Engenharia de Software II

Organização:

Este trabalho pode ser realizado em grupos de no máximo 5 integrantes.

Objetivos:

O trabalho final da disciplina consiste em um projeto extensionista na área de Tecnologia da Informação com o parceiro Nubo Desenvolvimento de Software LTDA (<https://nubo.ai/>).

O objetivo geral é projetar e implementar um MVP de acordo com a seguinte descrição.

Título

Plataforma de Agentes Conversacionais com Infraestrutura de Microsserviços

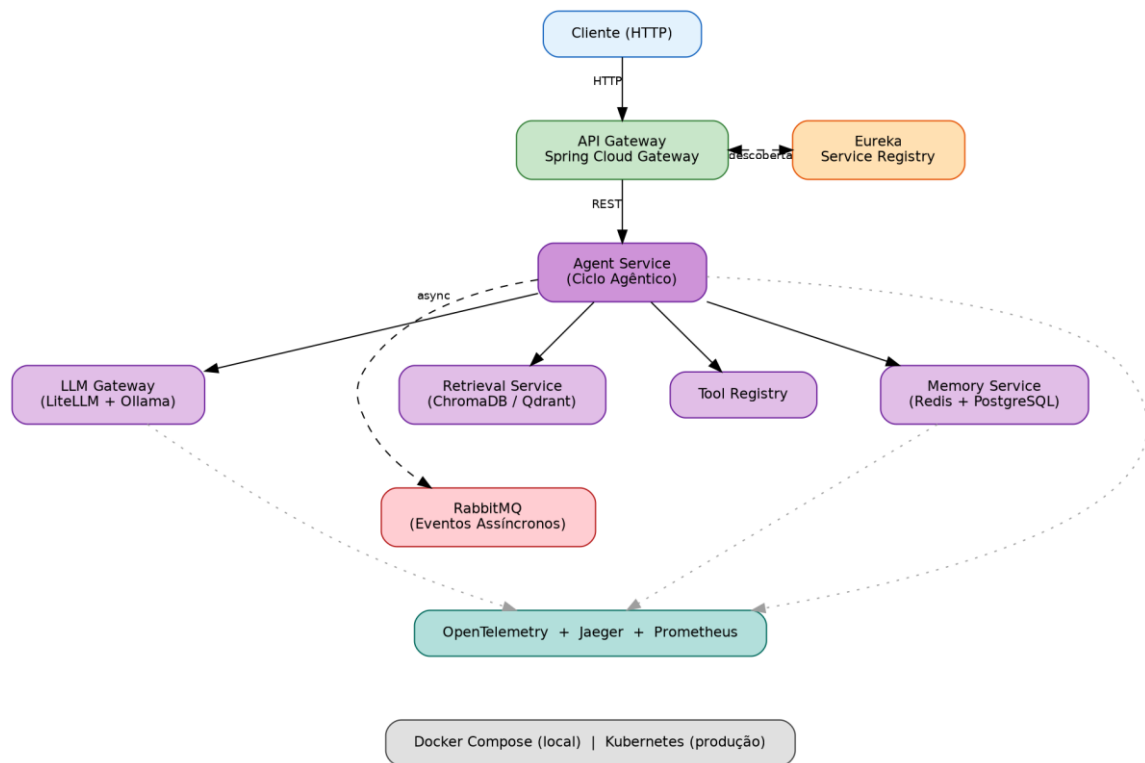
Descrição

O presente projeto tem como propósito projetar e implementar uma plataforma para execução de agentes de IA conversacionais, capaz de executar tanto em ambiente local por meio de Docker Compose quanto em produção em nuvem sobre um cluster Kubernetes. A execução local não deve depender de recursos de nuvem.

A plataforma será composta por microsserviços independentes que, em conjunto, constituem uma infraestrutura capaz de orquestrar agentes de IA, gerenciar memória de conversação, rotear requisições e prover resiliência.

No contexto deste projeto, um agente de IA é um sistema que opera em um ciclo contínuo de **raciocínio → ação → observação**: ele recebe uma requisição do usuário, raciocina sobre como resolvê-la por meio de um modelo de linguagem (LLM), pode invocar ferramentas externas para obter informações ou executar ações, observa os resultados obtidos e repete o ciclo até produzir uma resposta final. A plataforma fornece a infraestrutura necessária para que esses agentes operem de forma escalável, resiliente e observável.

A arquitetura proposta é composta pelos seguintes microsserviços:



#	Microsserviço	Responsabilidade	Tecnologia sugerida
1	agent-service	Recebe requisições do usuário, orquestra chamadas ao LLM e ferramentas, e mantém o ciclo agêntico (raciocínio → ação → observação)	Spring Boot ou Python (FastAPI)
2	llm-gateway	Proxy unificado para modelos de linguagem locais; abstrai o provedor de LLM para os demais serviços	LiteLLM + Ollama
3	memory-service	Armazena e recupera o histórico de conversação (memória de curto e longo prazo)	Spring Boot + Redis (curto prazo) + PostgreSQL (longo prazo)
4	retrieval-service	Fornece busca semântica em documentos (RAG — Retrieval-Augmented Generation)	Python (FastAPI) + ChromaDB ou Qdrant
5	tool-registry	Registra e expõe ferramentas que os agentes podem invocar (por exemplo: calculadora, consulta a base de dados)	Spring Boot

#	Microserviço	Responsabilidade	Tecnologia sugerida
6	api-gateway	Roteamento, rate limiting e ponto de entrada único para todos os serviços	Spring Cloud Gateway
7	name-server	Descoberta de serviços para todos os microserviços	Eureka Server

O RabbitMQ atua como message broker para comunicação assíncrona entre os microserviços, desacoplando produtores e consumidores de eventos. Os principais usos previstos são: publicação assíncrona de eventos de telemetria pelo Agent Service (chamadas ao LLM, tempo de resposta, ferramentas invocadas), sem bloquear a resposta ao cliente; enfileiramento de requisições de ingestão de documentos para indexação no Retrieval Service, permitindo processamento no ritmo do consumidor; e notificações entre agentes em cenários de múltiplos agentes, sem necessidade de esperar resposta.

Ao longo do projeto, os participantes deverão investigar e aplicar padrões de arquitetura de microserviços — comunicação síncrona e assíncrona, descoberta de serviços, roteamento via gateway, resiliência com circuit breaker, containerização e orquestração — integrando-os em uma solução funcional. Espera-se também a adoção de práticas de experimentação, incluindo análise de trade-offs, avaliação de desempenho e interpretação crítica dos resultados.

Justificativa

Visão de Negócio

Plataformas agênticas representam a fronteira atual de aplicações de IA, com demanda crescente. A arquitetura de microserviços viabiliza escalabilidade e desenvolvimento independente por equipes. A execução local elimina custos de nuvem durante o desenvolvimento.

Visão Acadêmica e Experimental

Problema real de engenharia de software que abrange decomposição de domínio, comunicação entre serviços, resiliência e observabilidade de forma integrada. Os participantes tomam decisões de projeto e as justificam tecnicamente, analisando compromissos entre desempenho, complexidade e viabilidade.

Resultados esperados

Ao final do projeto, espera-se a entrega de:

- a) Diagrama de arquitetura documentando os microserviços, suas responsabilidades e os protocolos de comunicação utilizados.
- b) Implementação funcional dos microserviços executando em containers Docker orquestrados via Docker Compose, incluindo:
 - Agent Service com ciclo agêntico funcional (ao menos uma chamada ao LLM e uma ferramenta);
 - LLM Gateway com Ollama servindo modelo de linguagem local;
 - Memory Service com persistência de histórico de conversação;
 - API Gateway com roteamento e circuit breaker;
 - Descoberta de serviços via Eureka.

- c) Comunicação assíncrona via RabbitMQ para ao menos um fluxo (por exemplo: ingestão de documentos, registro de eventos ou notificação entre agentes).
- d) Circuit breaker configurado e demonstrado (por exemplo: quando o LLM Gateway encontra-se indisponível, o agent-service responde com mensagem de fallback adequada).
- e) Observabilidade básica com rastreamento distribuído (OpenTelemetry + Jaeger) ou métricas (Prometheus + Grafana).
- f) Relatório técnico contendo decisões de arquitetura e respectivas justificativas, análise de trade-offs e dificuldades encontradas.
- g) Análise de evolução que discuta como a arquitetura implementada poderia ser adaptada para um cenário de produção em nuvem, identificando os componentes que necessitariam de substituição ou escalabilidade adicional.
- h) Descrição das alterações necessárias para que a solução passe a executar em produção em nuvem sobre um cluster Kubernetes, considerando que a execução local não depende de recursos de nuvem.

Referências

- Ollama — Execução local de modelos de linguagem: <https://ollama.ai>
- LiteLLM — Proxy unificado para modelos de linguagem: <https://github.com/BerriAI/litellm>
- ChromaDB — Banco de dados vetorial: <https://www.trychroma.com/>
- Qdrant — Banco de dados vetorial: <https://qdrant.tech/>
- Spring Cloud Netflix (Eureka) — Descoberta de serviços: <https://spring.io/projects/spring-cloud-netflix>
- Spring Cloud Gateway — API Gateway: <https://spring.io/projects/spring-cloud-gateway>
- Resilience4j — Circuit breaker: <https://resilience4j.readme.io/>
- RabbitMQ — Message broker: <https://www.rabbitmq.com/>
- OpenTelemetry — Observabilidade: <https://opentelemetry.io/>
- Jaeger — Rastreamento distribuído: <https://www.jaegertracing.io/>
- Docker — Containerização: <https://www.docker.com/>
- Kubernetes — Orquestração de containers: <https://kubernetes.io/>

Entrega:

A data limite de entrega do trabalho é 03/07/2026.

Entregas sugeridas

	Entrega
1 — Fundação	Microserviços básicos comunicando via REST (agent-service e llm-gateway com Ollama), executando localmente sem containers

	Entrega
2 — Infraestrutura	Eureka (name server) + API Gateway + circuit breaker configurados; serviços registrados e acessíveis por meio do gateway
3 — Memória e RAG	memory-service (Redis + PostgreSQL) e retrieval-service (ChromaDB) integrados ao agent-service
4 — Mensageria	RabbitMQ integrado para fluxos assíncronos (por exemplo: ingestão de documentos, telemetria)
5 — Containerização	Dockerfiles para cada serviço e arquivo docker-compose .yaml orquestrando toda a plataforma
6 — Observabilidade	OpenTelemetry + Jaeger; pipeline de integração contínua
7 — Produção em nuvem	Descrição das alterações e artefatos necessários para execução em cluster Kubernetes
8 — Entrega final	Relatório técnico; apresentação final com demonstração da plataforma em funcionamento através de vídeo disponibilizado de forma “não-listada” no Youtube
	• Repositório Git contendo todo o código-fonte, arquivos de configuração e instruções detalhadas de como iniciar a plataforma localmente e realizar testes. • Relatório apresentando metodologia, experimentos e conclusões. • Discussão apresentando oportunidades de melhoria e riscos identificados na plataforma, abrangendo aspectos como segurança, performance, escalabilidade e disponibilidade. • Apresentação final demonstrando a plataforma e discutindo os trade-offs das abordagens.